

Fingerprint Recognition System

Om Ghag^{1*}, Anurag Kumbhare^{1†}, Sarvesh Gaonkadar^{1†},
Tejas Kumbhar^{1†}, Sneha Annappanavar^{1*†}

¹Department Of Computer Engineering, Vidyalankar Institute of
Technology, Mumbai University, Wadala, Mumbai, 100190,
Maharashtra, India.

*Corresponding author(s). E-mail(s): omghag2003@gmail.com;
sneha.annappanavar@vit.edu.in;

Contributing authors: anuragkumbhare2043@gmail.com;
sarveshdgaonkadar6@gmail.com; tejaskumbhar8686@gmail.com;

[†]These authors contributed equally to this work.

Abstract

Convolutional Neural Networks (CNNs) have been a key framework for image-related tasks such as fingerprint recognition systems since the advancements in deep learning and computer vision. Traditional methods such as Scale Invariant Feature Transform (SIFT), Oriented FAST, Rotated BRIEF (ORB), and CNNs are used to match fingerprints. However, in recent years, studies have shown that CNNs are hard to work with large and complex datasets. Although accurate, SIFT is computationally expensive, while ORB is fast but not very precise.

Due to these challenges, the Siamese Neural Networks (SNNs) have been used for fingerprint matching tasks. In contrast to traditional methods, SNNs learn the similarity between input fingerprints, which are good for verification and matching tasks. It is demonstrated that using SNNs can lead to improved fingerprint pattern recognition in the presence of noise or partial prints, and they can even distinguish genuine from false fingerprints.

SNN based fingerprint recognition system offers the advantages of pairwise instead of global comparison and more accurate and reliable fingerprint matching. SNNs have been designed for local extraction and matching of features, however, given their light structure, they can be utilized on mobile security applications that require resource efficiency. High efficiency is of great importance for mobile security systems with limited computational resources.

We evaluate the application of SNNs to mobile fingerprint recognition systems from a comprehensive point of view to better understand the mobile security context where SNNs can potentially be applied. The results show that for real

time biometric authentication on mobile devices, SNNs are a good compromise between accuracy and computation, hence, are a suitable solution. This study validates the ability of SNNs to enhance the performance and security of mobile based fingerprint recognition systems, in a time when there is increasing demand for secure and efficient mobile biometric systems.

Keywords: Fingerprint Recognition, Siamese Neural Networks (SNNs), Convolutional Neural Networks (CNNs), Deep Learning in Biometrics, Mobile Security

1 Introduction

Convolutional Neural Networks (CNNs) have been used mostly in feature extraction and pattern recognition tasks in fingerprint recognition systems. They have made strong performance on multiple biometric applications possible because of their ability to capture local spatial hierarchies in images. However, CNNs are inefficient in terms of computational efficiency and handle global contextual information, which are important for mobile security environments with restrictions regarding computational resources.

Fingerprint recognition utilizes this to tackle the problems by the use of Siamese Neural Networks (SNNs) in mobile security applications. SNNs are particularly well suited to tasks like fingerprint matching and verification because they can be formulated as a twin network architectures that learn similarity metrics between two inputs. In a challenging biometric environment, SNNs also offer additional advantage of having greater accuracy and robustness in comparing pairs of fingerprint images to determine whether the produced images are from the same individual.

SNNs are particularly useful for mobile security, since fingerprint verification and most other biometric comparisons involve pairwise comparisons. Unlike traditional CNNs, SNNs have been built specifically to compare pairs of images at a time, and results confirm that they are robust to fingerprints that contain noisy or missing fingerprints. Additionally, SNNs have the right resource constrained nature appropriate to mobile platforms which permits computational demand to take the place of accuracy.

The research reported in this paper demonstrates how SNNs architecture can address the specific problems of mobile security, and specifically in the context of mobile fingerprint recognition systems. Given high accuracy and speed, SNNs are excellent for mobile device biometric authentication systems, and this allows fingerprint recognition systems to utilize SNNs for both goals.

2 Literature Survey

Fingerprint recognition systems have evolved from conventional Convolutional Neural Networks (CNNs) to more complex Siamese Neural Networks (SNNs). The purpose of these advances is to drive biometric systems to higher accuracy, greater efficiency, and increased robustness ([Rathi and Sawarkar \(2013\)](#)). In this review, an overview of

key SNNs and CNNs developments is provided, and their application to fingerprint recognition in mobile applications.

2. 1. Fingerprints: Fingerprint patterns are probably the most relied upon and widely used biometric identifiers for security purposes due to their uniquely unalterable patterns and ease of acquisition. The fingerprints of a person are unique compared to any other identical twins, as these patterns develop in the fetal stages of human development and remain invariant across a lifetime. Due to the inherent distinctiveness and permanence, fingerprints prove to be a good metric for identity verification and security applications (Chen and Wang (2016)).

2. 2. Advantages of Fingerprint Authentication over Traditional Security Systems as a Security Measure: Fingerprint authentication is a highly reliable biometric identification technique on the basis of uniqueness and cardinality of fingerprint, easy way of capture and permanence of fingerprint. Fingerprint authentication demonstrates better security than traditional authentication methods such as passwords or PINs because it provides these benefits:

- **Uniqueness & Immutability:** The unique pattern of fingerprint ridge structures together with finger bifurcations prevents identical prints between any two human beings. The fingerprint characteristics never change over someone's lifetime without requiring any updates making them immune to alterations. The system design ensures storage immortality of enrolled fingerprints since deletion is not possible which makes them excellent for extended security operations (Gune and Khedkar (2016)).
- **Universality & User-Friendliness:** Many different organizations including financial institutions together with law enforcement agencies and smartphone producers use fingerprint authentication worldwide because it offers universal access while maintaining user-friendly design. The authentication method operates rapidly with unobtrusive nature using standard optical sensors and scanners available to the market. The security of fingerprints represents a superior alternative to passwords since they resist all forms of hacking attempts and sharing methods.
- **Resistance to Attacks:** Fingerprints stand out as the strongest defense against biometric attacks among all forms of biometrics. Both facial recognition authentication and voice recognition can be defeated through image and video fraud and recorded voice inputs respectively. Fingerprint systems include factor detection features which stop spoofing activities.
- **Multi-Dimensional Security:** Fingerprint authentication provides multi-dimensional security because it examines fingerprint ridge patterns and minutiae points which makes the authentication mechanism much more secure than single-dimensional password systems. Brute-force attacks stand no chance against fingerprint-based systems because their high entropy levels protect them. The prediction of fingerprints turns out to be an extremely difficult task because of their elaborate nature.
- **Convenience & Compliance:** Users select simple passwords because they want convenience while simultaneously bypassing important security features including two-factor authentication. The simple biometric fingerprint scan system has minimal requirements and therefore leads to better user compliance and lower security risks.

- **Secure Local Matching & Encryption:** Fingerprint data on current devices stays within the device framework which stops the information from reaching outside systems. However from the privacy and security point of view they facilitate else encryption of the authentication systems as is the case with Apple's Touch ID and others ([Documentation \(2025\)](#), ([Bakhshi and Veisi \(2019\)](#))).

Fingerprint authentication is a better alternative to a traditional security system, solving the problems of security, usability, and resisting to attack by offering a robust and more efficient protection to modern applications.

2. 3. Fingerprint Matching Key Features: Appropriately chosen features from fingerprints are extracted and form a biometric template to enable comparison and matching for authentication or identification purposes in fingerprint security systems.

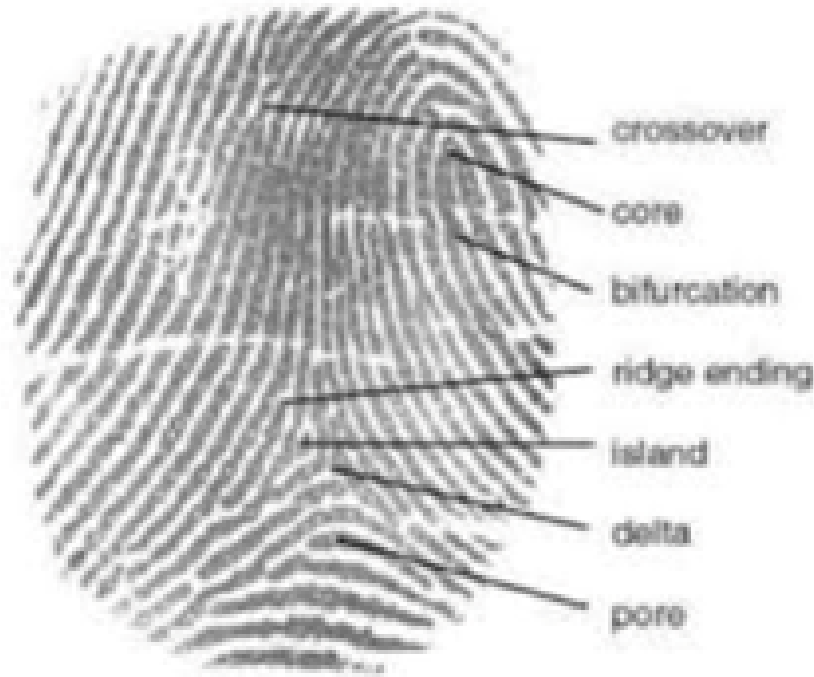


Fig. 1 Fingerprint Key Features

- **Ridge Patterns:** Fingerprints contain ridge patterns in loops, whorls, and arches, mainly due to their usage for classification and basic differentiation.
- **Minutiae Points:** The minutiae points, including ridge endings and bifurcations, primarily help systems identify and match partially captured fingerprints.
- **Ridge count and orientation:** In this feature, the ridges are counted at minutiae points for accurate matching in fingerprints. The orientation of ridges is compared for better results. The core is the center of a fingerprint, where the ridges form

loops or spirals, while the delta is formed by ridges, which split in a triangular fashion. Such reference points will guide the alignment of the fingerprint images to be compared, especially if the images have been rotated or distorted.

- **Pores and Sweat Glands:** Other fingerprint systems account for microscopic sweat gland pores, which strengthens security and increases accuracy in some applications ([Documentation \(2025\)](#)).

2. 4. Deep Learning for Biometric Recognition: The advantage of biometric recognition with deep learning is the ability for the extraction of high dimensional features of raw data. In particular, Convolutional Neural Networks (CNNs) have been already applied a great degree in text-based and image-based classification tasks.

For instance, CNNs were used to recognize misleading headlines in online media where they test features to distinguish click bait headlines from true content. The authors have used CNNs to conclude that they enable the detection of clickbait on social media platforms with accuracy of 82%. It outperformed the traditional machine learning such as Random Forest by focusing model on word sequence information and its semantic relationships.

This is also in line with fingerprint recognition where CNNs are used to learn ridge patterns, minutiae points, and structural relationships in fingerprint images. CNNs and their ability to learn complex textual dependence are just as useful for a clickbait detection task as they are in the accuracy of authentication learning unique fingerprint features. Feature extraction and classification using handcrafted features does not compare favourably with a CNN-based fingerprint recognition. This capability to generalize across different data modalities indicates the versatility of CNNs to be used in biometric security applications ([Shaikh and Annappanavar \(2020\)](#)).

2. 5. Convolutional Neural Networks (CNNs): CNNs have become the center of advancements in image recognition, from fingerprint recognition to image recognition, because they are capable of recognizing spatial patterns and hierarchies of images.

Major contributions in this area are: AlexNet (2012): The AlexNet was introduced in the world of image classification by Krizhevsky et al, that brought a lot of improvements using deep layers, ReLU and drop out. In large scale image datasets, it proved successful, and it has established itself as an important milestone in CNN development, beyond biometric tasks such as fingerprint recognition. Despite the success of CNNs in feature extraction, they remain constrained in the pairwise comparison tasks that are key to fingerprint matching and are consequently employed for fingerprint matching using Siamese Neural Networks (SNNs) ([Bakhshi and Veisi \(2019\)](#)).

2. 6. Siamese Neural Networks (SNNs): The advantage of SNNs is that they are specially made for tasks where there is a need to measure the similarity between two inputs, as in biometric matching systems such as fingerprint recognition.

Fingerprint Recognition with SNNs (2017): SNNs have been used on fingerprint recognition tasks where the model learns to compare two fingerprint images and learn a similarity function. A robust matching of fingerprints is provided by this method, in the case where the prints are noisy or incomplete. The selection of SNNs' architecture through which pairwise comparison is performed is particularly well suited for mobile fingerprint recognition systems due to the fact that new fingerprints can be compared

against stored biometric templates in an efficient manner ([Barzut and Milosavljević \(2022\)](#)).

2. 7. Application of SNNs in Mobile Fingerprint Matching: As compared to conventional SNNs, implementation of SNNs to fingerprint recognition in mobile devices presents several advantages, such as low computational overhead and high accuracy.

- **Pairwise Matching:** The twin architecture of SNNs allows the system to achieve high accuracy in matching tasks by comparing pairs of fingerprints, by attenuating the differences between fingerprint images.
- **Efficiency:** A variety of SNNs are optimized for mobile platforms, which provides efficient performance without sacrificing accuracy, an important property for mobile biometric systems
- **Robustness:** The advantage of SNNs is that they are able to handle noisy or partial fingerprint data, increasing system robustness in real world mobile applications where fingerprint scans may not be the best, and making it easier to design for such situations (([Zahid et al \(2024\)](#)), ([B and Sripriya \(2021\)](#)), ([Berdich et al \(2023\)](#))).

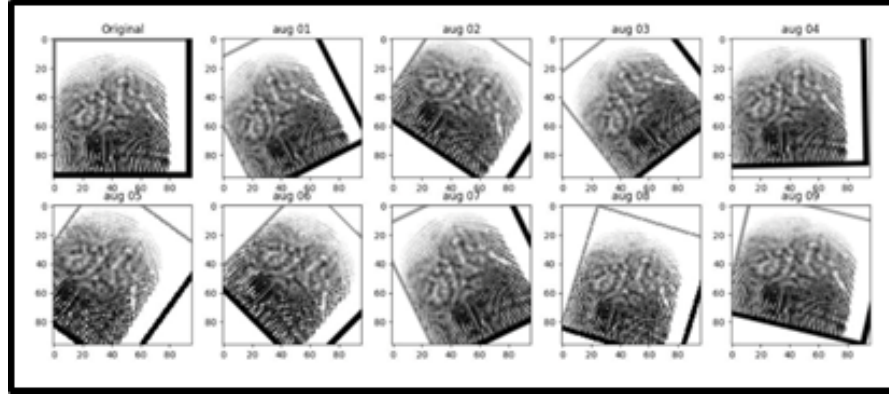


Fig. 2 Fingerprint Augmentation

3 Methodology

A structured approach to developing the Siamese Neural Network (SNN) for mobile fingerprint recognition is presented, beginning with real-time data collection, preprocessing, model training, encryption, mobile deployment and evaluation of performance.

3. 1. Data Collection and Preprocessing: Fingerprint data is collected in real-time when the user first registers their identity, and credentials on the mobile device in this system. When this registration phase takes place, the fingerprint image is captured and safely stored by AES encryption. The foundation of the user's biometric profile is based on this fingerprint data ([Afah et al \(2022\)](#)).

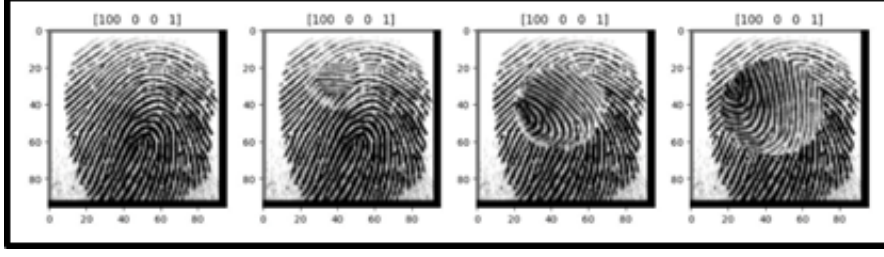


Fig. 3 Fingerprint Data Collection

Preprocessing steps are essential to maintain consistency and quality in the input data before training the model:

- **Image Resizing:** All fingerprint images are resized into a fixed size of 96x96 pixels. The standardization that comes from this size also reduces computational resources and speeds up processing time because this size is small enough. Smaller images, as the system is more efficient on mobile devices, most of them have limited processing power.
- **Normalization:** Once resized, pixel values are scaled between 0 and 1. This last step makes the model converge faster in training and adds the model's ability to generalize under a variety of lighting conditions, or fingerprint quality.
- **Data Augmentation:** During training, the use of data augmentation techniques increases the robustness of the model. Rotation, flipping, and translation are techniques such as these which aid a SNN to be able to recognize fingerprints in different orientations or in different conditions. It is important in dealing with the variability inherent in real world applications ([Saponara et al \(2021\)](#)).

3. 2. Model Architecture and Training: The SNN model is built with convolutional layers, max pooling layers and dense layers. The architecture is designed to extract distinctive features from fingerprint images and generate embeddings for comparison.

- **Training Objective:** A contrastive loss function is used to train the model, minimizing the distance between embeddings of matching fingerprints and maximizing the distance between nonmatching pairs. For the pairwise comparison tasks, this is an ideal approach, and SNN also accurately distinguishes genuine and impostor fingerprints during the authentication process
- **Hyperparameter Tuning:** The model performance is fine-tuned on key hyperparameters such as learning rate, batch size and the number of epochs. To prevent overfitting, early stopping is performed, so that once the model sees new, unseen data, it generalizes well ([Lone et al \(2022\)](#)).
- **Encryption and Secure Storage:** In order to guarantee sensitive biometric information security, the user's fingerprint data is stored at the time of registration, using AES encryption. During a login attempt, the encrypted fingerprint data is used for future comparison. A comparison between the pre-stored fingerprint and the user input fingerprint is done, and after the comparison, the data is processed in

line with the mobile device’s security protocols to match data integrity and privacy (Hammad et al (2024)).

4 Design

4. 1. Data Pre-processing Flowchart: This flowchart describes the critical steps for the preparation of biometric input data for processing within the model.

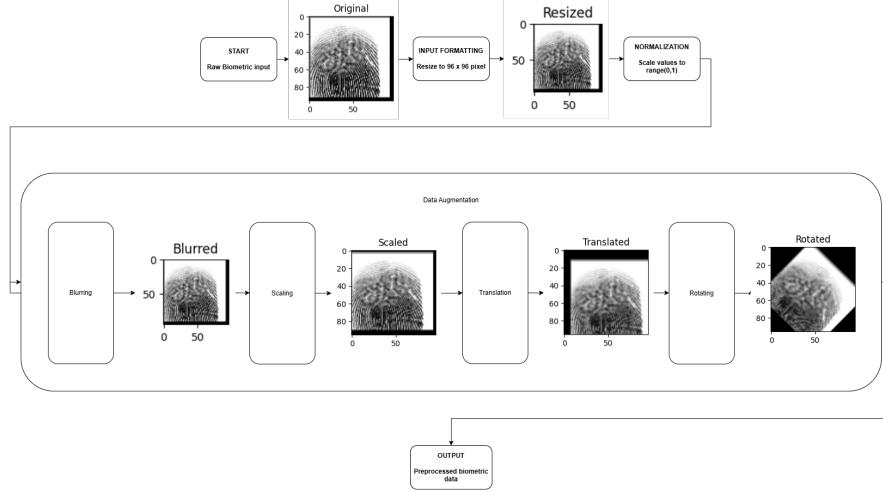


Fig. 4 Data Pre-processing Flowchart

- **Start (Raw Biometric Input):** Biometric data, specifically, fingerprint images are captured at the raw level, and after that the process starts.
- **Input Formatting (Resize to 96 x 96 Pixels):** The raw input images are then resized to a common dimension of 96x96 pixels to standardize the image size to help ease efficient processing. Along with this, the computational load is reduced and the resizing guarantees consistency in the data set.
- **Normalization (Scale Values to Range (0,1)):** All images are normalized to the range from 0 to 1 in terms of pixel values. Scaling improves the training efficiency and achieves higher consistency of the model input to learn from the data.
- **Data Augmentation (Apply Flipping, Rotation, Translation):** Different augmentation techniques are used to change the images, such as flipping, rotating and translating in order to enlarge the training dataset. In this approach, the model generalizes better and is more robust to the variation of fingerprint presentation.
- **Output (Pre-processed Biometric Data):** To a certain extent, the biometric data is rendered suitable for model input by following these preprocessing steps; therefore, this biometric data can be further processed and analysed (Hammad et al (2024)).

4. 2. Model Architecture: The architecture of the Siamese Neural Network used for fingerprint recognition is detailed in this flowchart which shows its multistage design.

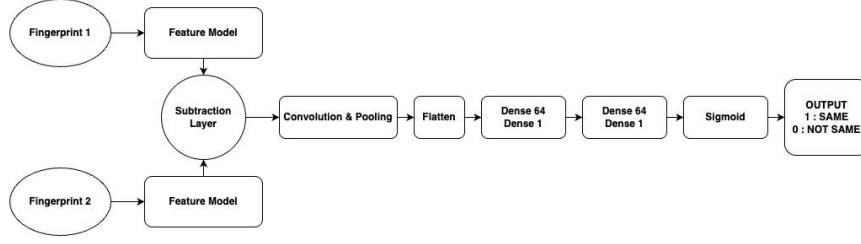


Fig. 5 Model Architecture Flowchart

- **Two Inputs (Fingerprint 1 and Fingerprint 2):** An architecture is presented, which starts with two different inputs, each corresponding to a fingerprint. First, the user's profile (previously registered fingerprint) provides one input, and the second input is captured when the user attempts to login. The setup allows us to compare biometric data.
- **Feature Model:** Each fingerprint is processed by a specialized feature extraction model that extracts salient features necessary for good comparison.
- **Subtraction Layer:** A subtraction layer turns into feature representations of both inputs, and they are compared using it. That allows the model to work with the differences in fingerprints.
- **Convolution and Pooling:** The resultant differences are processed through a series of convolutional layers followed by pooling operations. At this stage, there is a need to extract hierarchical features whilst simultaneously keeping dimensionality low, in order to enable the model to find relevant patterns.
- **Flatten:** The flattened processed features are then fed to feed to the following dense layers.
- **Dense 64, Dense 1:** The flattened vector is then passed through fully connected (dense) layers. The first dense layer with 64 neurons is intended to capture complex feature interactions, and the second, dense layer contains a single neuron which outputs a single prediction score.
- **Sigmoid:** Finally, a final dense layer feeds to an application of a sigmoid activation function to transform raw output into a probability score indicating the probability of match.
- **Output (1: final output binary (Same, 0: Not Same):** The final output is binary, a value of 1 means match (inputs belong to the same person), a value 0 means do not match (inputs do NOT belong to the same person) ([Guan et al \(2025\)](#)).

4. 3. Security Architecture: The secure fingerprint data processing flow using a Siamese Neural Network (SNN) for fingerprint matching in an Android device context is shown in this diagram. It highlights the key security items that protect biometric

data from the time of capture to the time of processing, and ultimately to the time of storage.

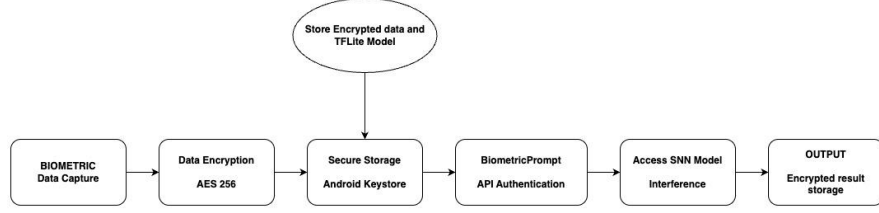


Fig. 6 Proposed Security Architecture Flowchart

- **Biometric Data Capture:** The process of security begins with capturing biometric data which is fingerprints via the device sensors. This initial stage is very important to make sure that the input data is handled securely.
- **Data Encryption (AES-256):** The biometric info is captured and after that the biometric information goes through AES-256 encryption, which is considered one of the most robust encryptions. This step in fact protects us from exposure of sensitive biometric data raw form transmission, storage, or via unauthorized access.
- **Secure Storage (Android Keystore):** And finally, in the Android Keystore is the encrypted biometric data and the TFLite model used for inference. This component ensures that cryptographic keys stay contained within the device, so that defense against unauthorized tampering and access to encrypted fingerprint data and the machine learning model used to learn who is who is strengthened.
- **BiometricPrompt API Authentication:** Before further processing it relies on the BiometricPrompt API to check for user authentication. This API makes it easy for a secure authentication flow using the biometric capabilities of the device, e.g., fingerprint or facial recognition, to be used only with those who are allowed to activate the SNN model for fingerprint matching.
- **Access SNN Model for Inference:** The system retrieves the Siamese Neural Network (SNN) model for inference upon successful authentication. The fingerprint comparison process with this model is highlighted with the secure TensorFlow Lite model and the input fingerprint is run through it to compare the stored biometric data to make a similarity determination and produce a matching score.
- **Output (Encrypted Result Storage):** The SNN inference produces a result, i.e. does it match or not, which is securely stored. The matching decision is encrypted and stored in a storage location that limits access by only authorized personnel, leaving the confidentiality of the matching decision undisturbed. It protects the results from possible data breach or unauthorized exploitation ([Guan et al \(2025\)](#)).

4. 4. Mobile Deployment Flow: The deployment of a trained Siamese Network model (SNN) for fingerprint authentication on a mobile device is illustrated by this flowchart.

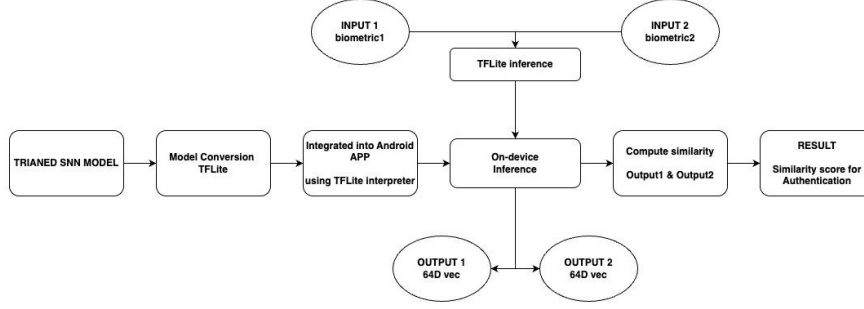


Fig. 7 Proposed Mobile Deployment Flowchart

- **Trained SNN Model (TensorFlow):** Using TensorFlow framework provides robust tools to build and optimal deep learning models, the process starts with SNN model training.
- **Model Conversion (TensorFlow Lite):** The model is then trained after which the model is converted into TensorFlow Lite (TFLite) format. It is crucial for the model to be able to operate efficiently on mobile devices to achieve performance optimality and accuracy at the same time.
- **Integrate into an Android App Using TFLite Interpreter:** Finally, the converted model is used in an Android application using the TFLite interpreter to do on device inference. It reduces the reliance on external servers, and it improves user privacy.
- **Input1 and Input2 (Biometric 1 and Biometric 2):** For comparison, the model takes two biometric inputs (fingerprints). The first input is taken from the user's profile (previously registered fingerprint), while the second input is captured when the user is trying to log in. With this setup, the user's authentication data can be directly analyzed.
- **TFLite Inference:** The on-device inference engine is used to process the inputs and generate embeddings, generating 64 dimensional vectors which represent the feature sets of the input fingerprints.
- **Output1 and Output2 (64-D Vector):** Outputs are two 64 dimensional vectors, one for each biometric input, which contain the essential features for comparison.
- **Compute Similarity:** Between the two output vectors, a similarity measure is computed, which measures degree of similarity between the fingerprints and aid in authentication decisions.
- **Result (Similarity Score for Authentication):** The system calculates the similarity score and based on the similarity score decides if the two fingerprints come from the same person. If the score is high, there is a match, and authentication occurs; if low, there is a mismatch ([Chen et al \(2022\)](#)).

5 Analysis

Figure A: The relationship between time per matching and similarity score: The relationship between two important variables in the matching process is illustrated in this scatter plot. **Key Observations:**

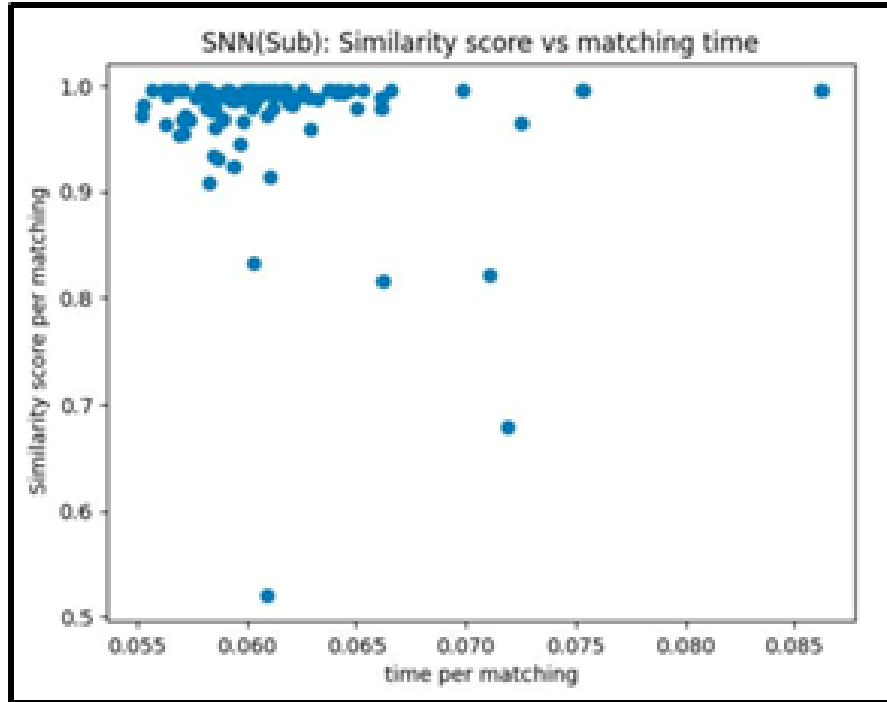


Fig. 8 Figure A

- X-axis (Time per Matching in seconds): Usually indicates the time needed to do a matching operation, e.g. nearest neighbours or similar matching algorithm.
- Y-axis (Similarity Score per Matching): It is a measure of similarity between two entities in terms of matching process, from score of 0 to 1, where 1 is the maximal similarity.

Analysis:

- The plot most of the data points cluster in the top left area, around a similarity score of 1 and time per matching around 0.06 seconds. This means that a large number of matching operations are both very accurate (similarity close to 1) and very fast (small time per matching).
- Lower similarity scores (between 0.5 and 0.9) with different matching times (between 0.065 and 0.085 seconds) are shown in a few outlier points. These cases correspond to situations where matching took longer or produced less similarity ([Chen et al \(2022\)](#)).

Figure B: SNN (Subtract) Model Train and Test Accuracy Across 100 Epochs: Here the comparison of train and test accuracy for a Spiking Neural Network (SNN) model with subtract operation over 100 epochs is plotted in this graph.

Key Observations:

- X-axis (Epochs): Number of training iterations, from 0 to 100.
- Y-axis (Accuracy): Accuracy is measured from 0.70 to 1.0 (70% to 100%).

Analysis:

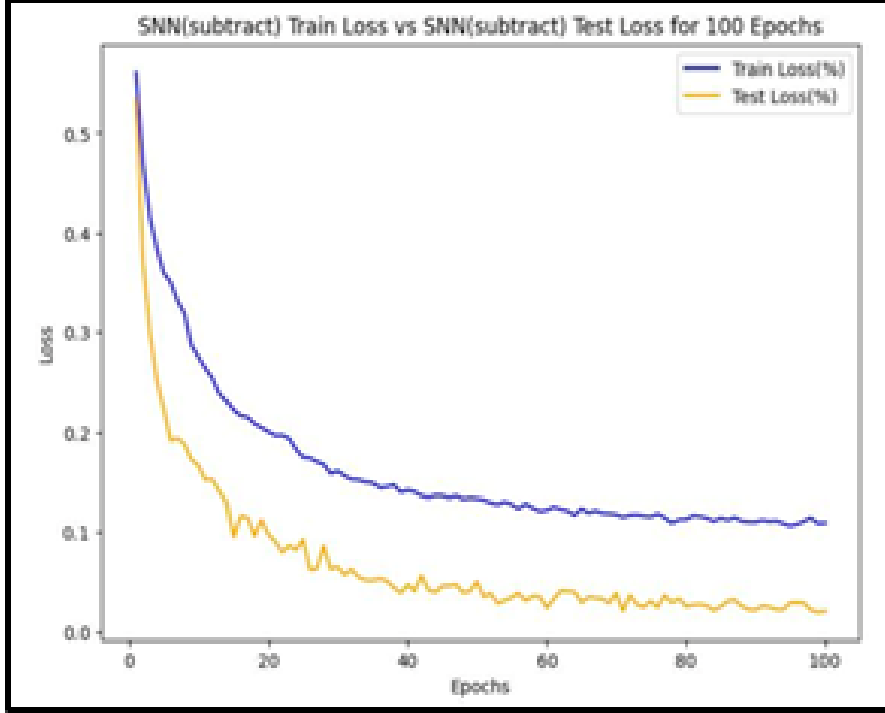


Fig. 9 Figure B1

- The blue line represents the train accuracy of the SNN (subtract) model for which the accuracy begins at approximately 70% and increases up to a little above 96% at the end of training indicating effective learning.

- The orange line represents test accuracy, starting at a comparable value, increasing more quickly to almost 98% after the 20th epoch and then staying at a stable level. Generalization to unseen data shows strong evidence of minimal overfitting and this trend ([Chen et al \(2022\)](#)).

Figure C: Using SNN(L1): Similarity Score vs Matching Time: Here similarity scores are compared against matching time of a fingerprint recognition system based on the Siamese Neural Network (SNN) with L1 distance.

Key Observations:

- Y-axis (Similarity score per matching): It is a measure of how similar two fingerprint templates are (0.55 to 0.90).
- X-axis (Time per matching): It shows the time taken for each matching from 0.055 to 0.090 seconds.

Analysis:

- Most matches fall into the top left (higher similarity scores, faster matching times), with some points close to a maximum similarity score (0.90) at low times (0.055 to 0.065 seconds).
- At slightly higher matching times (above 0.065 seconds) there are some outliers with lower similarity scores (around 0.55-0.75).

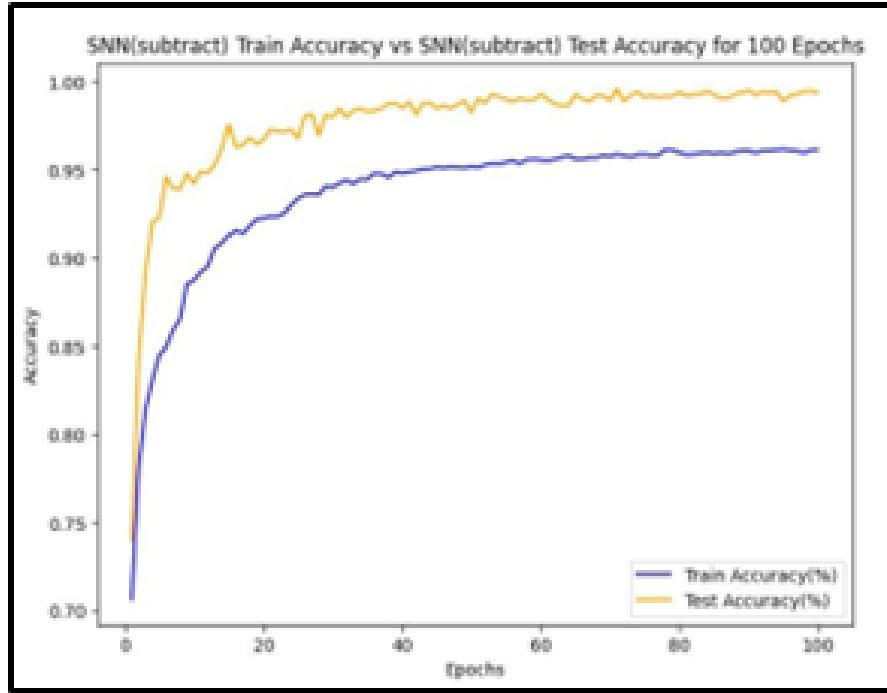


Fig. 10 Figure B2

· A clear trade-off between similarity scores at different time intervals, with the highest performance in both accuracy and time near 0.90 score mark (Shaikh and Annappanavar (2020)).

Figure D: Training Accuracy Comparison of SNN Models: This line graph compares the training accuracy of two SNN (Siamese Neural Network) models over 15 epochs.

Key Observations:

- Y-axis (Accuracy): Represents the accuracy of each model during training.
- X-axis (Epochs): Number of epochs during training, ranging from 1 to 15.

Analysis:

· L1 SNN (Blue Line): Shows a gradual increase in accuracy, starting at around 60% and steadily climbing to approximately 78% over 15 epochs. This model improves consistently but at a slower rate.

· Subtract SNN (Yellow Line): Begins with a higher initial accuracy, around 83%, and reaches a peak of about 92% accuracy in the first few epochs. The improvement slows after 5 epochs but maintains a higher accuracy compared to the L1 SNN (Shaikh and Annappanavar (2020)).

Figure E: Test Accuracy Comparison of SNN Models over 100 Epochs: This line graph compares the test accuracy of two SNN (Siamese Neural Network) models over 100 Epochs.

Key Observations:

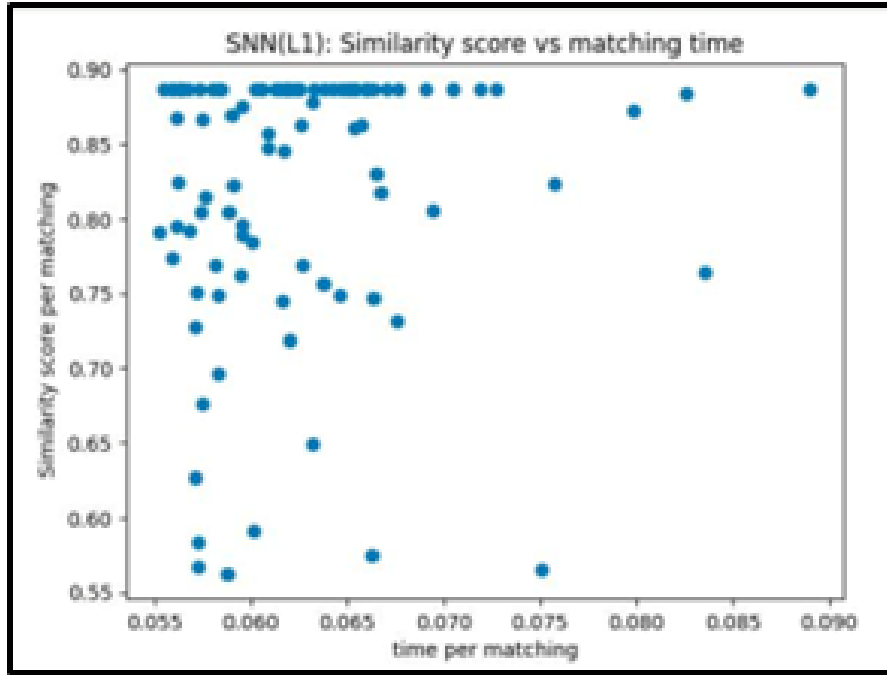


Fig. 11 Figure C

- Y-axis (Accuracy): Represents the test accuracy of each model.
- X-axis (Epochs): Number of epochs during the testing phase, ranging from 1 to 100.

Analysis:

- L1 SNN (Blue Line): The test accuracy of the L1 model remains fairly consistent, fluctuating between 55% and 60% throughout the epoch. There is no significant improvement over time.
- Subtract SNN (Yellow Line): The Subtract SNN model starts with a high accuracy of approximately 90% and shows slight improvements, peaking close to 98% before stabilizing (Shaikh and Annappanavar (2020)).

Figure F: Comparison of Similarity Score of Models: In the comparison, comparison of the similarity scores of two SNN (Siamese Neural Network) models with different matching methods for the same matching time.

Key Observations:

- Y-axis (Similarity Score per Matching): This is the amount of how well each model can match the data. The closer the score is to 1, the stronger the match.
- X-axis (Time per Matching): It is the time taken by each model to match.

Analysis:

- SNN (Sub) (Blue Dots): This method shows high similarity scores consistently and most of the clusters are near 1.0, indicating strong performance. And the time per matching stays low and is concentrated around 0.06 seconds.

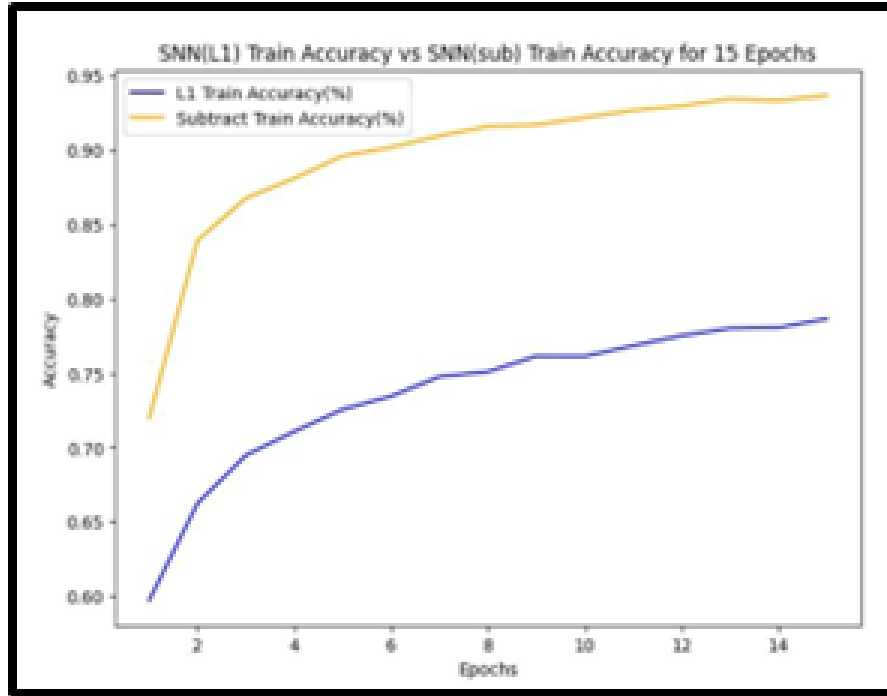


Fig. 12 Figure D

- SNN (L1) (Orange Dots): The similarity scores of this method are more spread out, with values ranging from 0.4 to 0.9 which exhibit more variance in performance. The matching times are more widespread, beyond 0.1 seconds.

Figure G: The Matching Rate Comparison of Models: The matching rates of three models are compared in a verification or classification task and this bar graph compares them.

Key Observations:

- Y-axis (Matching Rate %): It represents the accuracy of each model to identify or match data.
- X-axis (Models): It compares three different approaches.

Analysis:

- Traditional Minutiae-based (Gray Bar): It achieves a matching rate of about 60% which is relatively low.
- CNN-based (Orange Bar): It demonstrates a matching rate of around 85% which is a better accuracy compared with the traditional method.
- Siamese Neural Networks (SNNs) (Blue Bar): It achieves matching rates close to 100% and demonstrates the best performance of the models.

Figure H: Comparison of Storage Requirements of Models: The storage requirements for each of the three models are shown in this bar graph.

Key Observations:

- Y-axis (Storage Requirement in MB): It is the memory required per model.

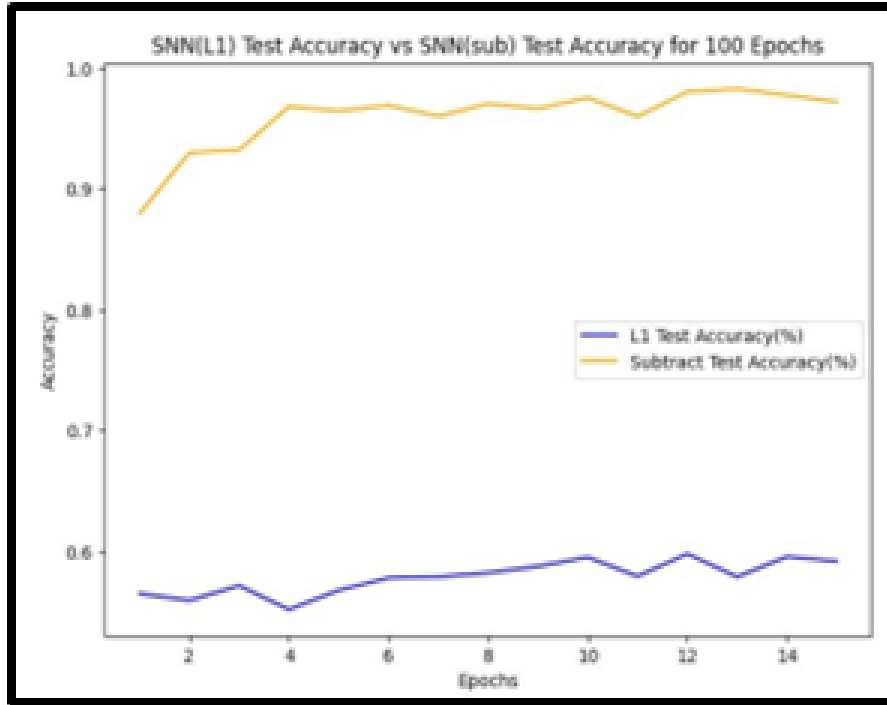


Fig. 13 Figure E

- X-axis (Models): The same three models are then compared.

Analysis:

- Traditional Minutiae-based (Gray Bar): It takes about 1 MB of storage, making it the least memory demanding option.
- CNN-based (Orange Bar): This means that the storage requirements are significantly higher than that.
- Siamese Neural Networks (SNNs) (Blue Bar): Strikes the balance between minutiae-based model and CNN based model, using around 3 MB.

Figure I: Speed Comparison of Models: Below is a graph depicting the time (in milliseconds) taken per match by each model.

Key Observations:

- Y-axis (Time per Match in ms): Indicates how long each model takes to perform a match.
- X-axis (Models): Compared to the following models:

Analysis:

- Traditional Minutiae-based Model: The time of shows the highest at around 40 ms which means it is one of the slowest to do among the options.
- CNN-based Model: One that is faster than the minutiae based (approximately 30 ms per match).
- Siamese Neural Network (SNN): With a time per match below 10 ms, the fastest model.

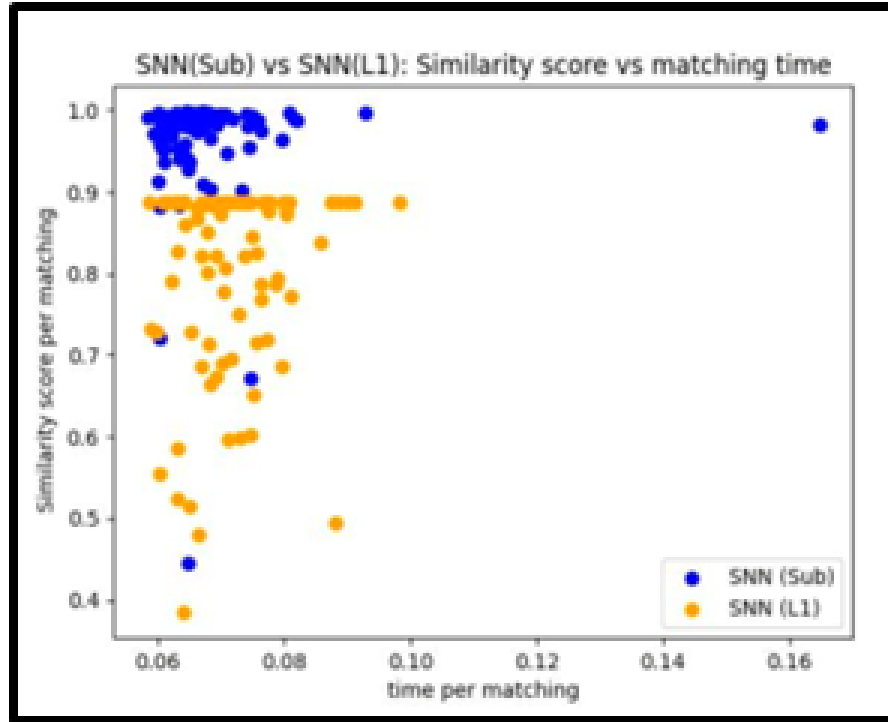


Fig. 14 Figure F

Figure J: Accuracy Comparison of Models: This graph shows the accuracy of each model, as a percentage.

Key Observations:

- Y-axis (Accuracy %): They represent the accuracy of each model.
- X-axis (Models): Compares them with the following models.

Analysis:

- Traditional Minutiae-based Model: It has an accuracy of around 70%.
- CNN-based Model: Approximately 90% accuracy shows improved.
- Siamese Neural Network (SNN): It achieves the highest accuracy, approximately 95%

Figure K: Comparison of Fingerprint-Matching Models in Computational Complexity: On a logarithmic scale, this graph compares the computational complexity of Traditional Minutiae based, CNN based, and SNN based fingerprint matching models using the number of Floating-Point Operations per Second (FLOPs).

Key Observations:

- Y-axis (FLOPs): Is the computational complexity represented on a logarithmic scale.
- X-axis (Models): It compares the following models.

Analysis:

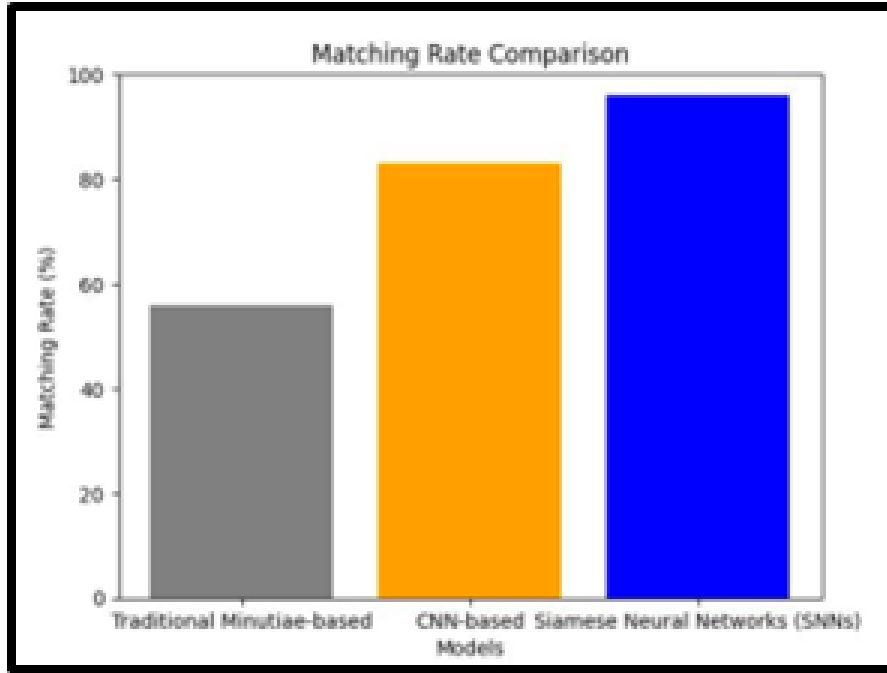


Fig. 15 Figure G

- Traditional Minutiae-based Model (Gray Bar): Has the lowest FLOPs, around 106. It is seen that this method demands much less computational resources than the other models.
- CNN-based Model (Orange Bar): The FLOPs are displayed at approximately 1011. This suggests that CNN based models are very computationally expensive and require a much higher number of operations to do fingerprint matching.
- Siamese Neural Network (SNN)-based Model (Blue Bar): Shows a FLOPs value around 1010. Still orders of magnitude above traditional minutiae-based approaches, which is slightly lower than CNN based models.

6 Conclusion

6. 1. Conclusion from fingerprint recognition of image: The fingerprint recognition image is made up of three panels that present the fingerprint recognition process in the context of biometric security. The different labels and scores show how the system evaluates the similarity between fingerprints:

1. Left Panel (Input Image): - Label: Input: [504 0 0 3] - It is an input fingerprint image on this panel. The fingerprint looks a bit rotated, an initial raw (or unprocessed image) that needs correction or normalization. The print is clear, ridge lines are visible, but there are some borders or artifacts around the edges of the image ([Rathi and Sawarkar \(2013\)](#)).

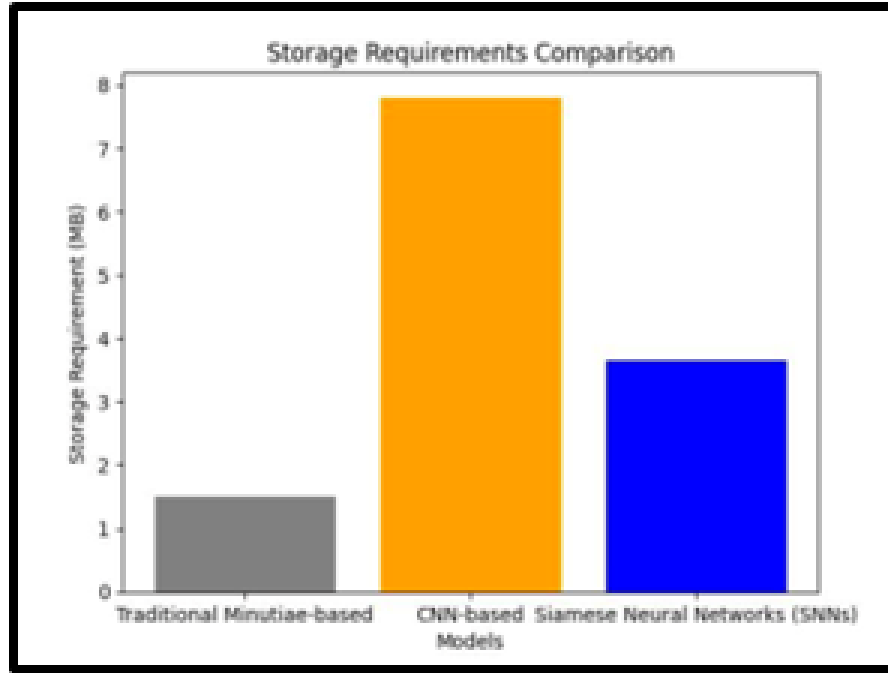


Fig. 16 Figure H

2. Middle Panel (Matched Fingerprint): - Label: O: 0.94, [504 0 0 3] - This panel has a high match score (0.94) making this is a successful matching attempt or recognition. The fingerprint pattern looks corrected or aligned with a well-centered clear fingerprint pattern. This score of 0.94 means that input fingerprint and a stored template are very similar (i.e., a success match) ([Chen and Wang \(2016\)](#)).
3. Right Panel (Unmatched Fingerprint): - Label: X: 0.00, [182 0 1 3] - As this score is 0.00, this is a failed matching attempt. Here the fingerprint seems darker and more distorted, and a few ridge lines seem less defined than in the middle panel. This fingerprint does not match because of the lower quality or distortion of it ([Gune and Khedkar \(2016\)](#)).
1. Left Panel (Input Image): Label: Input: [188 0 1 4] - Ridges in the fingerprint display a mild rotation which usually occurs in unprocessed images needing alignment or normalization. Smooth ridge lines run through the image while artifacts produce noticeable damage at its outer edges.
2. Middle Panel (Matched Fingerprint): Label: O: 0.94, [188 0 1 4] -The fingerprint success rate is validated through a high match score of 0.94. Well-defined ridge-line patterns merge with the improved alignment of the fingerprint along with a more central position.

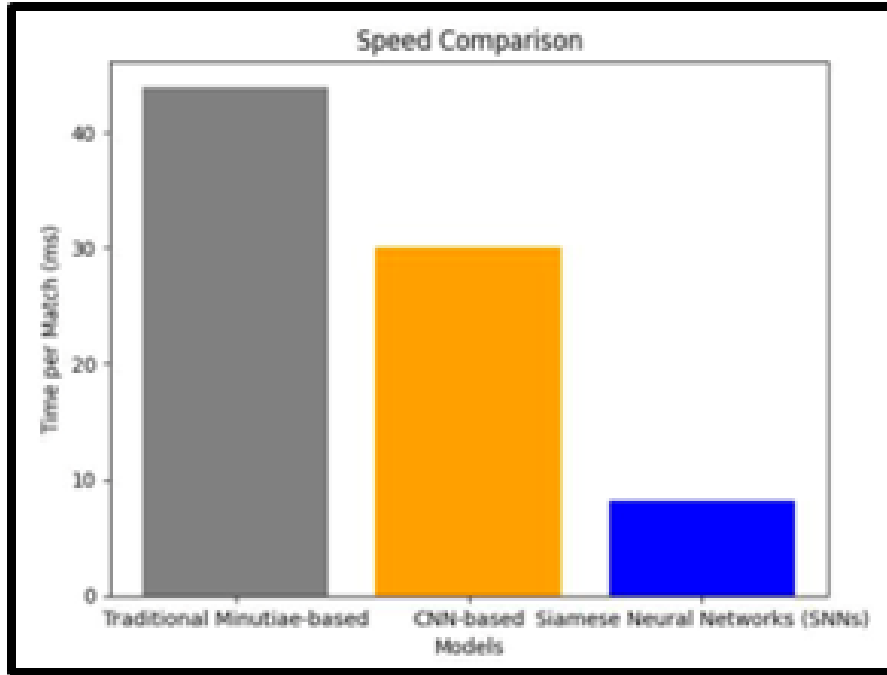


Fig. 17 Figure I

3. Right Panel (Unmatched Fingerprint): Label: X: 0.00, [363 0 0 2] - The analyzed fingerprint results in an evaluation score of 0.00 which represents unsuccessful identification. A blurry fingerprint pattern exists in this image while its ridge structure remains hazy in contrast to the reference fingerprint.
1. Left Panel (Input Image): Label: Input: [373 0 0 1] - The fingerprint shows a slight angle which suggests possible correction should be performed alongside normalization. The ridges are accessible for viewing yet particular segments display both noise along with distortion.
2. Middle Panel (Matched Fingerprint): Label: O: 0.97, [373 0 0 1] - The recognition attempt proves successful based on the high match score of 0.99. The fingerprint matches the template perfectly since it maintains proper alignment while clearly displaying its ridge patterns.
3. Right Panel (Unmatched Fingerprint): Label: X: 0.02, [472 0 1 1] - The matching procedure results in failure when the score stands at 0.00. A blurry and distorted fingerprint on the left panel indicates that the biological print is substantially different from what is stored on the database.
1. Left Panel (Input Image): Label: Input: [295 0 0 1] - Noise appears among areas in the rotated fingerprint print. The outlined shapes become harder to identify because the areas remain faint when scanned over darker components.

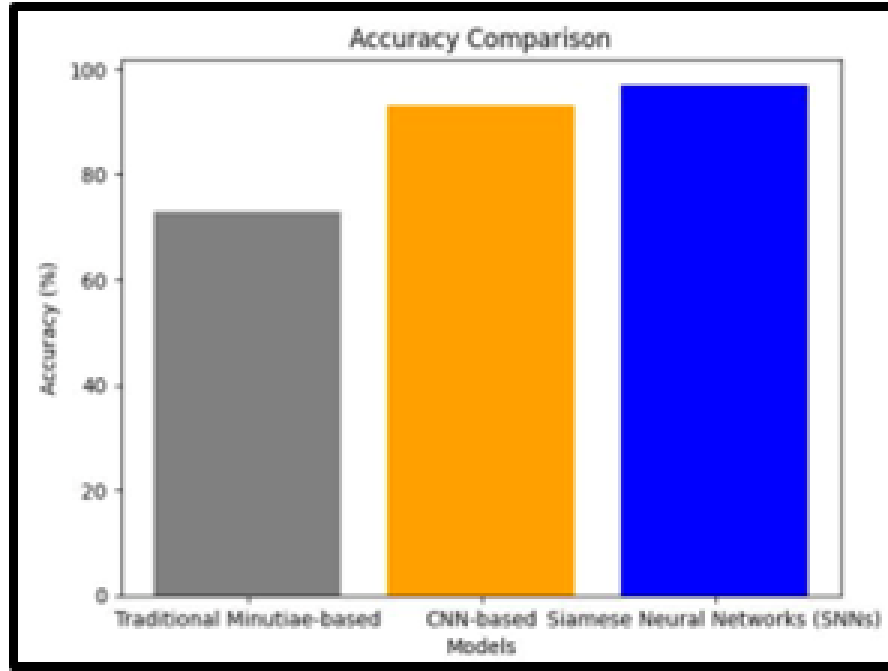


Fig. 18 Figure J

2. Middle Panel (Matched Fingerprint): Label: O: 0.99, [295 0 0 1] - With a match score reaching 0.97 the recognition meets extreme success standards. The fingerprint stands in good center position and matches perfectly with the recorded print.
3. Right Panel (Unmatched Fingerprint): Label: X: 0.00, [472 0 0 1] - The provided fingerprint exhibits no match with the reference profile according to the score of 0.02. Image distortion and lack of ridge detail in structure leads to this mismatch observation.

6. 2. Conclusion derived from the figures: The figures are used for an in-depth comparison of different fingerprint matching models including Siamese Neural Networks (SNNs), CNN based models and traditional minutiae-based methods.

1. Performance and Accuracy:
 - (a) The Siamese Neural Networks (SNNs) are nearly perfect with accuracy rates of nearly 100% and also display high similarity scores. For fingerprint recognition tasks, the SNN (Subtract) model outperforms the SNN (L1) model, and it is the most effective model ([Documentation \(2025\)](#)).
 - (b) As with traditional methods, CNN based models have accuracy rates of 85–90% but are dramatically slower and less efficient than SNNs. The matching rates of traditional minutiae-based models are lowest (around 60%-70%) and thus the least suitable for highly demanding fingerprint matching tasks ([Bakhshi and Veisi ,2019](#)).
2. Speed and Efficiency:

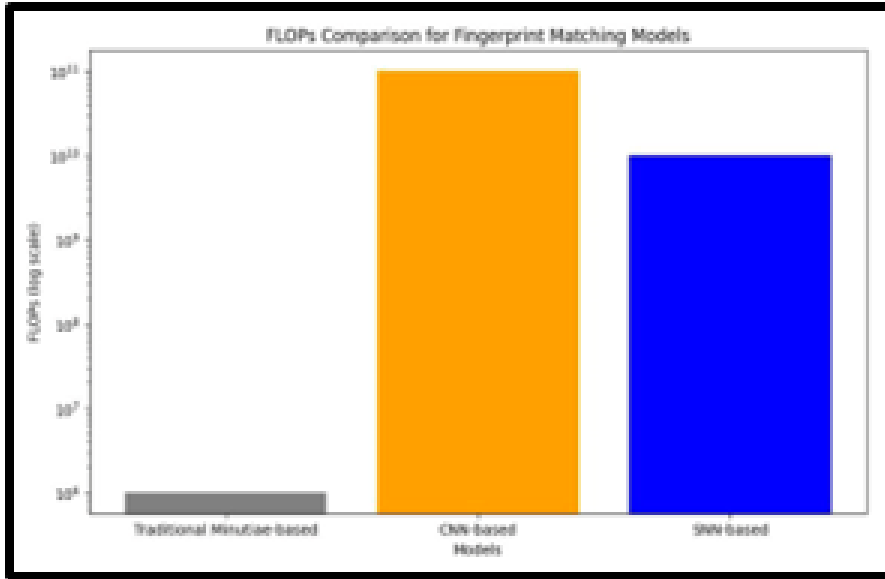


Fig. 19 Figure K

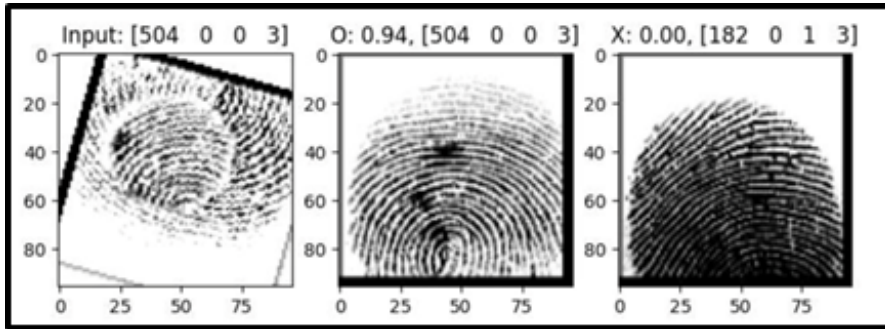


Fig. 20 Fingerprint Recognition Output1

- (a) The fastest matching times (below 10 milliseconds) make SNNs particularly well suited to real-time fingerprint recognition, for example, unlocking mobile devices or secure access points.
 - (b) SNNs are moderately slow but much more efficient than CNNs. The slowest to perform are traditional minutiae-based methods, which take about 40 milliseconds to match, which makes them impractical for use in real-time applications like mobile security ([Barzut and Milosavljević \(2022\)](#)).
3. Computational and Memory Efficiency:
- (a) SNNs come up with their good trade-off that uses less computational power than any CNN model and yet produces good performance. They also have moderate storage requirements which are suitable for devices with little resources, such as mobile devices.

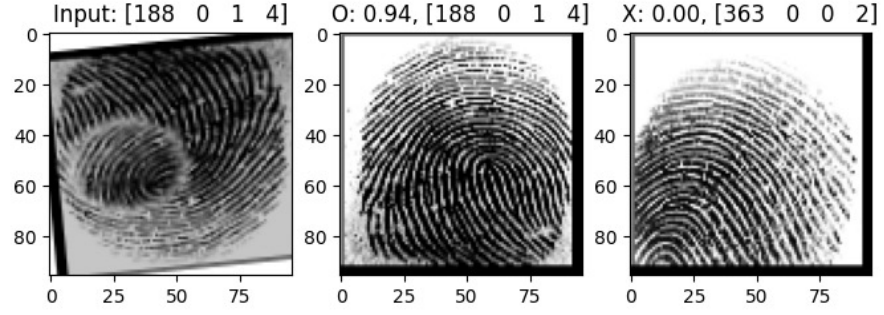


Fig. 21 Fingerprint Recognition Output2

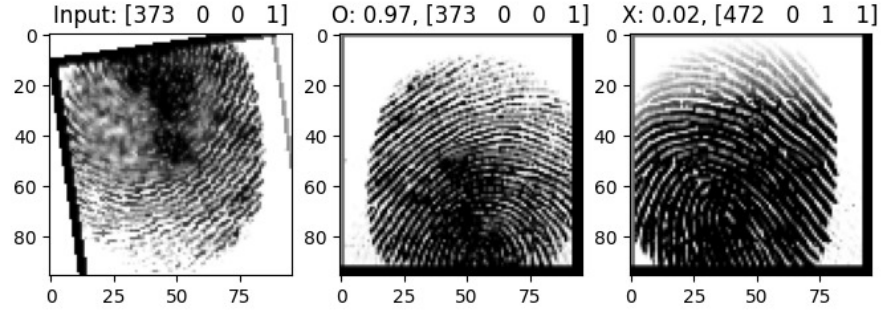


Fig. 22 Fingerprint Recognition Output3

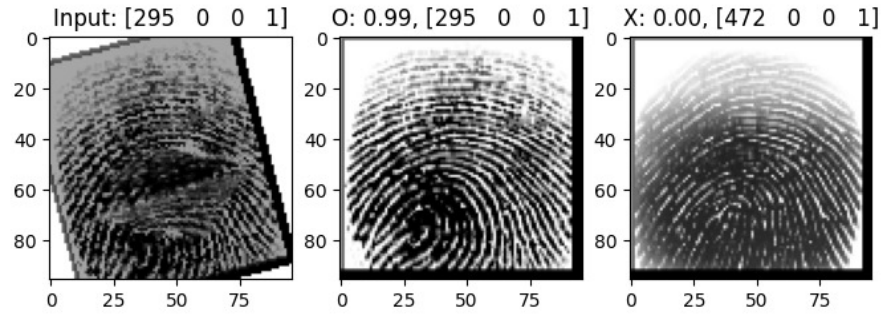


Fig. 23 Fingerprint Recognition Output4

- (b) CNN-based models are computationally expensive in terms of memory and processor requirement and may not be convenient for mobile devices constrained with hardware limitations. On the other hand, models based on traditional minutiae can make use of the least resources, However, their accuracy and speed are not high enough to be desirable in secure fingerprint recognition ([Zahid et al \(2024\)](#)).

4. Training and Generalization:

- (a) The SNN models learn exceptionally well and there is no overfitting on both training and unseen data. For instance, the SNN (Subtract) model obtains 98% test accuracy, which is very accurate for mobile applications.
- (b) SNN models generalize as well but are outperformed by the CNN models that are more efficient. Training and generalization are not evaluated by traditional methods, yet they are considered less adaptable to more complex tasks like fingerprint matching (B and Sripriya ,2021)).

6. 3. Implications for Mobile Fingerprint Security:

- Compared to other security systems, such as those based on keystroke dynamics, this analysis is particularly relevant for mobile fingerprint security systems that need to achieve high accuracy, fast response times, and efficient use of computational resources.
- Mobile security applications require quick and accurate fingerprint matching, and SNNs are the best choice for mobile security. They can achieve near perfect accuracy within milliseconds so that it doesn't impact the user experience and at the same time keeps the security strong (Berdich et al .,2023), (Afah et al .,2022)).
- Mobile use for CNN based models may be plausible but they would face a ceiling on energy consumption possibly because of greater computational requirements or memory, causing slower unlock times or battery drain (Saponara et al (2021)).
- The current mobile security approach requires models to function faster along with greater accuracy whereas traditional minutiae-based models offer slower speeds combined with lower accuracy which results in reduced user satisfaction (Lone et al (2022)).

Therefore, the conclusion is that Siamese Neural Networks (SNNs) are the most appropriate models for mobile fingerprint security due to the excellent balance between speed, accuracy and resource efficiency, compared to other proposed models. Thanks to their quick response time and high accuracy, mobile devices can be unlocked quickly and securely in a modern mobile environment (Hammad et al (2024), (Guan et al (2025), (Chen et al (2022), (Shaikh and Annapanavar (2020)).

Declarations

Funding: *Not Applicable*

Competing interests: *The authors declare no competing interests.*

Ethics approval and consent to participate: *Not Applicable*

Consent for publication: *Not Applicable*

Data availability: [Sokoto Coventry Fingerprint Dataset \(SOCOFing\)](#)

Materials availability: *Not Applicable*

Code availability: [FingerPrintMatchingSNN](#)

Author contribution: Om Ghag: Conceptualization, Building and fine-tuning of Model. Sarvesh Gaonkardkar: Documentation of the process and research, drafting manuscript. Anurag Kumbhare: Researched and analysed the problem statement and market needs. Tejas Kumbhar: Conceptulized model implementation. Sneha Annapanavar: Oversight and mentorship throughout the research process and critical revision of the manuscript for important intellectual content.

References

- [1] "Fingerprint Matching Algorithm for Android." International Journal of Engineering Research and Technology (IJERT), vol. 2, no. 10, 2013
- [2] Chen, Y., & Wang, H. "Efficient Fingerprinting-Based Android Device Identification with Zero-Permission Identifiers." ResearchGate, 2016
- [3] Gune, S., & Khedkar, S. "The Recognition of Fingerprints on Mobile Applications - An Android Case Study." ResearchGate, 2016.
- [4] "Fingerprint Hardware Abstraction Layer (HAL)." Android Developer Documentation.
- [5] B. Bakhshi and H. Veisi, "End to End Fingerprint Verification Based on Convolutional Neural Network," 2019 27th Iranian Conference on Electrical Engineering (ICEE), Yazd, Iran, 2019, pp. 1994-1998, doi: 10.1109/IranianCEE.2019.8786720.
- [6] S. Barzut, M. Milosavljević, "The Application of Convolutional Neural Networks for Fingerprint Recognition: A Comparative Analysis," in Sinteza 2022 - International Scientific Conference on Information Technology and Data Related Research, Belgrade, Singidunum University, Serbia, 2022, pp. 224-229. doi:10.15308/Sinteza-2022-224-229
- [7] Kamil, Bayda Zahid, Dhahir Abdulhadi Abdullah, and Haraa Raheem Hatem. "Biometrics based on deep learning: A survey." Int. J. Nonlinear Anal. Appl. (2024).
- [8] Nithya, B., and P. Sripriya. "Fingerprint Identification by Training a LSTM Network with Fingerprint Segments as Sequence Inputs." 2021 6th International Conference on Communication and Electronics Systems (ICCES). IEEE, 2021.
- [9] A. Berdich, B. Groza and R. Mayrhofer, "A Survey on Fingerprinting Technologies for Smartphones Based on Embedded Transducers," in IEEE Internet of Things Journal, vol. 10, no. 16, pp. 14646-14670, 15 Aug.15, 2023, doi: 10.1109/JIOT.2023.3277883.
- [10] Afah, Daudu, et al. "Smartphones verification and identification by the use of fingerprint." Advanced Techniques for IoT Applications: Proceedings of EAIT 2020. Springer Singapore, 2022.
- [11] S. Saponara, A. Elhanashi and Q. Zheng, "Recreating Fingerprint Images by Convolutional Neural Network Autoencoder Architecture," in IEEE Access, vol. 9, pp. 147888-147899, 2021, doi: 10.1109/ACCESS.2021.3124746.
- [12] Lone, Sajaad Ahmed, and A. H. Mir. "Smartphone-based biometric authentication scheme for access control management in client-server environment." International Journal of Information Technology and Computer Science (IJITCS) 14.4 (2022): 34-47.

- [13] M. Hammad, M. A. Wani, K. A. Shakil, H. Shaiba and A. A. A. El-Latif, "Deep Cancelable Multibiometric Finger Vein and Fingerprint Authentication with Non-Negative Matrix Factorization," in *IEEE Access*, vol. 12, pp. 120638-120660, 2024, doi: 10.1109/ACCESS.2024.3450372.
- [14] X. Guan, Z. Pan, J. Feng and J. Zhou, "Joint Identity Verification and Pose Alignment for Partial Fingerprints," in *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 249-263, 2025, doi: 10.1109/TIFS.2024.3516566.
- [15] S. Chen, Z. Guo, X. Li and D. Yang, "Query2Set: Single-to-Multiple Partial Fingerprint Recognition Based on Attention Mechanism," in *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 1243-1253, 2022, doi: 10.1109/TIFS.2022.3159151.
- [16] M. A. Shaikh and S. Annappanavar, "A Comparative Approach for Clickbait Detection Using Deep Learning," *2020 IEEE Bombay Section Signature Conference (IBSSC)*, Mumbai, India, 2020, pp. 21-24, doi: 10.1109/IBSSC51096.2020.9332172.